Auditoría de mantenibilidad y limpieza

Estado: auditoría y limpieza de los límites prioritarios cerradas. El reporte distingue la verificación realizada de la deuda residual; no afirma que cada archivo del repositorio necesite o haya recibido una refactorización.

Corte de evidencia (2026-10-02): cifras de líneas, conteos de pruebas y resultados de comandos abajo son el registro de la ejecución original de la auditoría, no una ejecución nueva en esta actualización documental. Las tareas de documentación no alteraron código ni cambian esa evidencia.

Diagnóstico

El principal problema no es la cantidad de líneas por sí sola, sino la mezcla de responsabilidades en archivos que coordinan reglas de negocio, acceso a datos, estado de interfaz y presentación. La medición excluye código generado y `dist`.

Archivo	Líneas antes de esta limpieza	Responsabilidades mezcladas
`apps/web/src/modules/orders/views/OrderDetailView.vue`	1.887	Consultas y mutaciones, formularios de unidades/equipos, reporte, medios privados, auditoría y modales.
`apps/api/src/modules/execution/execution.service.ts`	1.344	Ejecución de órdenes, checklist, medios privados, cierre y corrección.
`apps/api/src/modules/orders/orders.service.ts`	1.102	Creación y edición de órdenes, unidades, equipos, asignación y reprogramación.
`apps/web/src/modules/orders/views/OrderCreateWizardView.vue`	760	Orquestación del asistente y presentación de pasos; ya delega parte de la lógica en composables probados.

Archivo	Líneas antes de esta limpieza	Responsabilidades mezcladas
`apps/web/src/modules/service-execution/views/OrderExecutionView.vue`	752	Estado de ejecución, notas, medios, checklist y acciones de ciclo de vida.

La vista de detalle administrativo no tenía pruebas unitarias o de componente directas; `apps/web/e2e/orders.spec.ts` sí cubría sus puntos de entrada con API simulada. En backend existen pruebas extensas de órdenes y ejecución que deben proteger los invariantes durante futuras extracciones.

Limpieza aplicada

Unidad	Resultado	Evidencia
Medios privados del detalle	`useOrderMedia` concentra precarga, previsualización y liberación de URLs de evidencias y firma.	Tres pruebas enfocadas en precarga, reintento, previsualización y liberación.
Agregar unidad	`useAddOrderUnit` concentra estado, selecciones, validación y mutación; `AddOrderUnitModal` contiene su interfaz.	Cinco pruebas del composable; prueba de navegador para apertura del modal.
Secciones del detalle	`OrderEvidenceTab`, `OrderSyncTab`, `OrderReportCard` y `OrderAuditTab` separan presentación mediante propiedades y eventos tipados.	Typecheck, build y prueba de navegador para las pestañas de evidencia/sincronización.
Notas de ejecución	`useExecutionNotes` centraliza persistencia, indicador de guardado y bandera de cambios sin mezclar el guardado manual con el cierre.	Tres pruebas del composable, incluidos errores de persistencia.
Reprogramación de cita fallida	`reschedule-failed-appointment.ts` separa resolución, clonación de unidades/equipos y auditoría de `OrdersService`.	Pruebas de una transacción, orden de escrituras, conflicto de versión y snapshots inactivos.
Finalización de órdenes	`OrderFinalizationService` concentra completitud, cierre, cita fallida y corrección; `ExecutionService` conserva la API pública y delega.	Pruebas existentes de estados, precios y outbox; prueba nueva del fallo de encolado del PDF posterior al commit.
Procesamiento de evidencias	`image-preprocessing.ts` separa conversión HEIC/HEIF y redimensionado JPEG de la UI y del protocolo de carga.	Cinco pruebas de dimensiones, formato, metadatos y errores de lectura/decodificación.
Reintentos de evidencias Drive	`evidence-upload-retry.ts` encapsula la clave de idempotencia, su persistencia de sesión y el respaldo en memoria; el	Cinco pruebas de identidad, persistencia, prioridad en

Unidad	Resultado	Evidencia
	adaptador de objetos conserva su flujo independiente.	memoria, almacenamiento inaccesible y limpieza.
Galería de evidencias	`useEvidenceGallery` administra cargas, liberación de URLs privadas y respuestas tardías al eliminar evidencias o cambiar de orden/unidad.	Cinco pruebas de carga, retirada, cambio de contexto, borrado y desmontaje.
Asignación de equipo	`useAttachOrderEquipment` y `AttachOrderEquipmentModal` separan el formulario y su mutación del detalle; conservan el payload original.	Cuatro pruebas de permisos, reinicio, mutación y errores.
Medios de ejecución	`ExecutionMediaService` concentra evidencias y firmas; `ExecutionService` mantiene su API pública y delega. Drive y el adaptador de objetos siguen rutas explícitas.	Pruebas existentes de Drive y dos nuevas para confirmación y errores del proveedor de objetos.
Almacenamiento de reportes	`report-storage.ts` separa reserva/reutilización en Drive y escritura del proveedor de objetos de la carga de datos y transacción READY/auditoría.	Pruebas de reserva concurrente, desaparición del PDF remoto y clave/checksum del proveedor de objetos.
Pestaña de unidades	`OrderUnitsTab` separa la presentación mediante propiedades y eventos tipados; el detalle conserva mutaciones, permisos y modales.	Prueba de navegador del evento Quitar hasta DELETE y apertura de Asignar Equipo.
Consultas de órdenes	`OrdersQueryService` concentra listado, detalle administrativo/técnico y auditoría; `OrdersService` conserva delegados públicos compatibles.	Cuatro pruebas nuevas de acceso denegado y lecturas sin consultas innecesarias.

`OrderDetailView.vue` pasó de **1.887 a 764 líneas** (-1.123, -59,5 %); `OrderExecutionView.vue` pasó de **752 a 744**; `orders.service.ts` pasó de **1.102 a 791** (-311, -28,2 %); `execution.service.ts` de **1.344 a 396** (-948, -70,5 %); y `EvidenceCapture.vue` de **456 a 322**. `ReportsService` quedó en **357 líneas**. En el árbol actual no queda ningún archivo de producción TypeScript/Vue de más de 1.000 líneas si se excluyen pruebas, código generado, `dist` y artefactos de CDK. La reducción refleja límites de responsabilidad: no se comprimió código para alcanzar una cifra. Los nuevos módulos siguen siendo archivos revisables por separado.

Verificación actual

- `pnpm typecheck`, `pnpm lint` y `pnpm build`: correctos.
- `pnpm test`: 25 pruebas de contratos, 310 de API y 124 de web correctas; 6 pruebas de API permanecen omitidas.
- Playwright `e2e/orders.spec.ts` en Chromium de escritorio: 16 pruebas correctas, incluidas las acciones de la pestaña de unidades y la navegación por las pestañas extraídas.
- Prettier en los archivos modificados y `git diff --check`: correctos.

- `pnpm format:check` global: falla en 13 archivos **no modificados** por esta limpieza. Es deuda de formato previa, no un fallo atribuido a los cambios de esta unidad.
- Build web: conserva el aviso de un chunk mayor de 500 kB; esta limpieza no abordó tamaño de bundles.

Duplicación y límites de abstracción

- `useExecutionNotes` eliminó la persistencia duplicada de notas; el refresco y mensaje de éxito siguen siendo exclusivos del guardado manual.
- `useAddOrderUnit` eliminó los restablecimientos duplicados del formulario de unidad sin introducir un framework genérico de modales.
- `orders.service.ts` repite parte de la validación de disponibilidad de equipos dentro de `assignEquipment` y `assertEquipmentAvailable`. Los mensajes de error actuales difieren; cualquier unificación debe conservar el contrato observable y la protección de unicidad en la base de datos.
- **No abstraer por inercia:** `useOrderMutations` ya centraliza la invalidación de caché. Drive y el adaptador de objetos tienen protocolos de carga, idempotencia y errores distintos; un wrapper genérico ocultaría esas diferencias.
- La firma Drive usa el mismo mecanismo básico de persistencia de claves que la evidencia, pero su identidad depende del firmante normalizado y del checksum, y vive en el módulo API. Compartir el mecanismo podría reducir duplicación; unir ambos flujos o sus identidades cambiaría sus contratos de reintento.

Límites conservados y deuda residual

- `OrderDetailView.vue` (764 líneas), `OrderCreateWizardView.vue` (760) y `OrderExecutionView.vue` (744) conservan orquestación y plantillas extensas. El asistente ya delega bloques, reemplazos y envío en composables; extraer secciones adicionales solo por tamaño trasladaría complejidad sin clarificar responsabilidades.
- `OrdersService` (791 líneas) conserva operaciones administrativas CRUD y asignación; las lecturas están en `OrdersQueryService` con delegados compatibles, sin introducir otro cableado en router/app. `assignEquipment` y `assertEquipmentAvailable` comparten validaciones, pero difieren en mensajes y contexto. No se unificaron sin caracterizar esos contratos y la restricción de unicidad de base de datos.
- `ExecutionMediaService` (600 líneas) mantiene juntas las operaciones de medios, pero conserva las diferencias entre Drive y el adaptador de objetos. Un split adicional por proveedor exigiría pruebas de paridad de permisos, reintentos y confirmación.
- La interfaz de carga de evidencias no tiene una prueba de navegador que ejecute de extremo a extremo ambos proveedores. Las pruebas unitarias, de router y de servicio protegen los límites refactorizados, pero no reemplazan una integración real con almacenamiento.
- Las cargas simultáneas desde ambos controles de `EvidenceCapture` no se serializaron: cambiar esa política sería comportamiento nuevo, no limpieza mecánica.
- `pnpm format:check` sigue fallando en 13 archivos preexistentes no tocados; formatear los HTML de salida generaría cientos de líneas de churn ajeno a esta limpieza. El chunk

`heic2any` aún supera 500 kB y las 6 pruebas API omitidas siguen siendo deuda independiente.

El cierre significa que los límites prioritarios identificados se separaron y verificaron, no que todo riesgo del sistema haya desaparecido. Los cambios quedaron sin commit para revisión del responsable del proyecto.